

Angular^{v20}



Component Lifecycle Hooks

Angular Component Lifecycle Hooks

Every Angular component goes through a **series of steps (hooks)** from creation to destruction.

Angular provides **lifecycle hook methods** that let you **run code at each stage**.

Constructor => Inject Services, initialize properties

Component Class Instantiated

```
↓
constructor()
↓
ngOnChanges() ← on every @Input change
↓
ngOnInit() ← after first ngOnChanges
↓
ngDoCheck() ← every change detection run
↓
ngAfterContentInit() ← content <ng-content> projected
↓
ngAfterContentChecked() ← content checked
↓
ngAfterViewInit() ← child views & @ViewChild ready
↓
ngAfterViewChecked() ← view checked
↓
(User interacts... data updates...)
↓
ngOnDestroy() ← before component is removed
```

constructor

ngOnChanges

ngOnInit

ngDoCheck

ngAfterContentInit

ngAfterContentChecked

ngAfterViewInit

ngAfterViewChecked

ngOnDestroy



Most Common Hooks (with Use Cases)

Hook	When It Runs	Use For
<code>ngOnInit()</code>	Once after component is created	Fetch data, init logic
<code>ngOnChanges()</code>	On each change to <code>@Input()</code> props	React to input changes
<code>ngAfterViewInit()</code>	After child views (e.g. <code>@ViewChild()</code>) init	Access DOM/child components
<code>ngOnDestroy()</code>	Before the component is removed	Clean up: unsubscribe, clear intervals

ts

Copy Edit

```
ngOnChanges(changes: SimpleChanges): void
```

- `SimpleChanges` is an object containing:

- `currentValue`
- `previousValue`
- `firstChange`



Slide Title: **When to Use** `constructor()` **vs** `ngOnInit()`



constructor()

The constructor is a **TypeScript feature**, not an Angular-specific one.



Use it for:

- Initializing basic variables
- Injecting dependencies (services, etc.)
- Setting up simple class logic



Do **not** use it to access `@Input()` values — they're not yet available.



ngOnInit()

`ngOnInit()` is an Angular **lifecycle hook**, called once the component is initialized and **input bindings are resolved**.



Use it for:

- Accessing `@Input()` data
- Fetching data from a service
- Executing code that depends on the component being fully ready

 Lifecycle Hook	 Description	 Real Use Cases
<code>ngOnInit()</code>	Called once after component is initialized	- Fetch API data - Initialize variables - Start subscriptions
<code>ngOnChanges()</code>	Called when <code>@Input()</code> values change	- React to parent input changes - Trigger logic based on new input
<code>ngAfterViewInit()</code>	Called after component's view and child views are initialized	- Access <code>@ViewChild</code> DOM elements - Initialize 3rd-party libraries
<code>ngOnDestroy()</code>	Called just before the component is destroyed	- Unsubscribe from observables - Clear intervals/timers - Cleanup

✓ Component Example: ngOnDestroy with setInterval

ts

Copy Edit

```
import { Component, OnInit, OnDestroy } from '@angular/core';

@Component({
  selector: 'app-timer',
  template: `
    <p>Timer: {{ counter }} seconds</p>
  `
})
export class TimerComponent implements OnInit, OnDestroy {
  counter = 0;
  private intervalId: any;

  ngOnInit(): void {
    // Start counting every second
    this.intervalId = setInterval(() => {
      this.counter++;
      console.log('Tick:', this.counter);
    }, 1000);
  }

  ngOnDestroy(): void {
    // Clear the interval when component is destroyed
    if (this.intervalId) {
      clearInterval(this.intervalId);
      console.log('Timer cleared!');
    }
  }
}
```



SIGNALS

Signals

What Are Signals?

Signals are a new way to manage reactive values in Angular — They **hold a value** and **automatically update the UI** or any code that depends on them when the value changes.

Why Use Signals?

-  Simpler than RxJS for UI state
-  No need to `subscribe()` or `unsubscribe()`
-  Works with zone-less Angular (better performance)
-  Perfect for **component state**

Signals

Syntax & Usage

ts

 Copy  Edit

```
import { signal } from '@angular/core';
```

```
// Create a signal
```

```
const count = signal(0);
```

```
// Read it
```

```
console.log(count()); // 0
```

```
// Set or update it
```

```
count.set(5);
```

```
count.update(n => n + 1);
```

Signals

◆ 2. `computed()`

Creates a **readonly** value that automatically updates when other signals change.

ts

 Copy  Edit

```
const count = signal(2);  
const double = computed(() => count() * 2);  
  
console.log(double()); // 4  
  
count.set(5);  
console.log(double()); // 10
```

✓ Great for derived state like `totalPrice = quantity * price`

Signals

◆ 3. `effect()`

Runs code whenever the signals it uses change.

ts

 Copy  Edit

```
const count = signal(0);

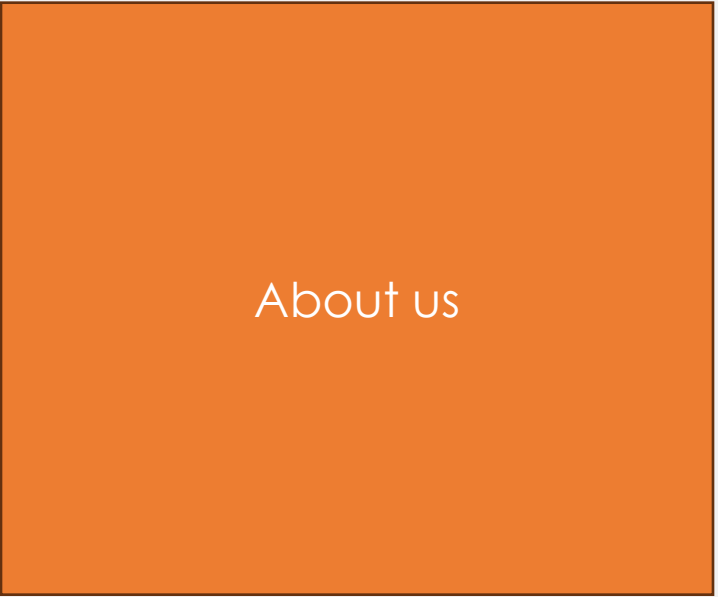
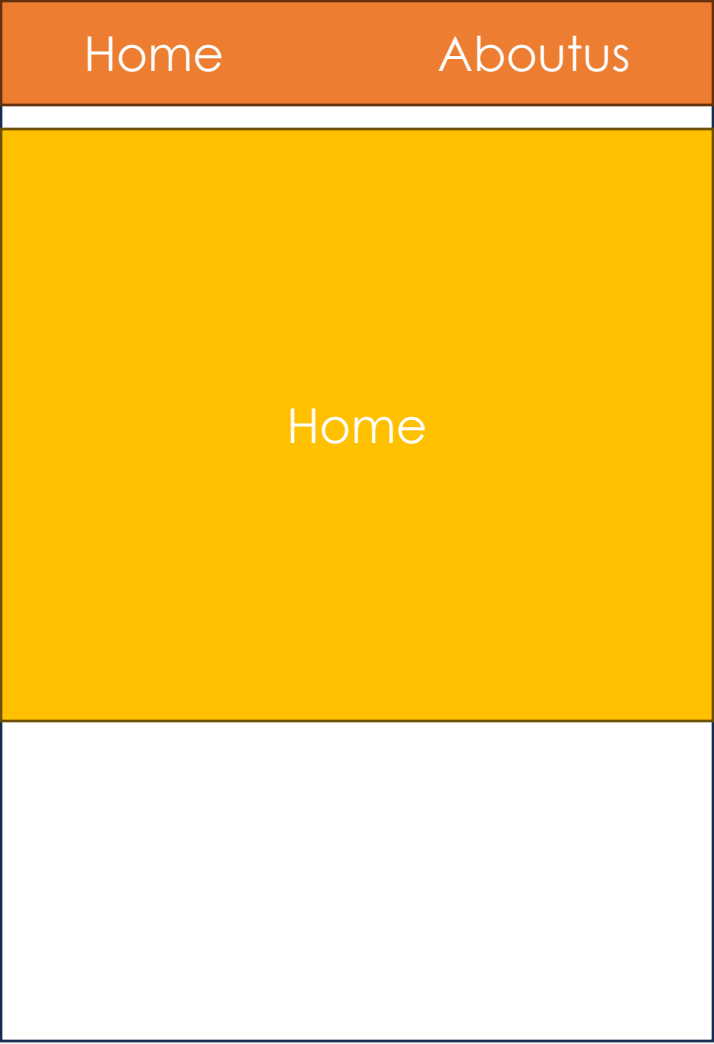
effect(() => {
  console.log('Count changed to', count());
});
```

Signals

Input,Output Function For Component Interaction

```
userNameInput = input<string>('No User Name Provided');  
onUserRemoved = output<number>();  
this.onUserRemoved.emit(1);
```

Routing



Routing



What is Routing?

Routing allows your Angular app to behave like a **Single Page Application (SPA)** — users can navigate between views without reloading the page.



Step 1: Set Up Routes

ts

Copy

Edit

```
// app.routes.ts
import { Routes } from '@angular/router';
import { HomeComponent } from './home.component';
import { AboutComponent } from './about.component';

export const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'about', component: AboutComponent },
  { path: 'user/:id', component: UserComponent },
  { path: 'optional-user/:id', component: UserComponent },
];
```

In app.config.ts:

ts

Copy

Edit

```
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes)]
};
```

Type	Example Path	Notes	
Default route	''	Loads HomeComponent	
Static route	'about'	Loads AboutComponent	
Parameterized route	'products/:id'	Loads details dynamically (e.g., /products/5)	
Nested (child) route	products/details/:id	Used for showing subcomponents inside a parent	
Protected route (guard)	'admin' with canActivate	Only if user is authorized	
Lazy-loaded module	'auth'	Module loaded only when path is accessed	
Wildcard route	'**'	Catch-all for 404s	



<router-outlet>

This is a **placeholder** in your template where Angular loads the routed component.

html

Copy Edit

```
<router-outlet></router-outlet>
```



routerLink

Binds a link to a route **without reloading** the page:

html

Copy Edit

```
<a routerLink="/about">About</a>
```

You can also pass route parameters:

html

Copy Edit

```
<a [routerLink]="['/user', 42]">User 42</a>
```



routerLinkActive

Adds a CSS class when the link is **active**:

html

Copy Edit

```
<a routerLink="/about" routerLinkActive="active-link">About</a>
```



ActivatedRoute

Used in a component to **read route parameters**:

ts

Copy Edit

```
import { ActivatedRoute } from '@angular/router';

constructor(private route: ActivatedRoute) {}

ngOnInit() {
  this.route.paramMap.subscribe(params => {
    const id = params.get('id');
    console.log('User ID:', id);
  });
}
```

`This.route.snapshot.paramMap`

2 Redirects and Wildcard Routes

Redirect empty path to `home`:

ts

 Copy  Edit

```
{ path: '', redirectTo: 'home', pathMatch: 'full' }
```

Catch-all 404 route:

ts

 Copy  Edit

```
{ path: '**', component: NotFoundComponent }
```

4 Programmatic Navigation with Router

Inject `Router` to navigate in code:

ts

 Copy  Edit

```
constructor(private router: Router) {}

goToUser() {
  this.router.navigate(['/user', 123], { queryParams: { edit: true } });
}
```

Routing Guards

What is a Route Guard?

A **Route Guard** is a function or service that tells Angular whether a route **can be accessed, loaded, or left**.

✓ Most common use: **authentication check before showing a route**

Common Guard Types

Guard Type	When It Runs
<code>canActivate</code>	Before navigating to a route
<code>canDeactivate</code>	Before navigating away from a route
<code>canLoad</code>	Before lazy-loading a module
<code>canActivateChild</code>	Before accessing child routes

Routing Guards

Step 1: Create the Guard

bash

Copy Edit

```
ng generate guard auth
```

Step 2: Implement Logic

ts

Copy Edit

```
// auth.guard.ts
import { Injectable } from '@angular/core';
import { CanActivate, Router } from '@angular/router';

@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  constructor(private router: Router) {}

  canActivate(): boolean {
    const isLoggedIn = false; // mock logic
    if (!isLoggedIn) {
      this.router.navigate(['/login']);
      return false;
    }
    return true;
  }
}
```

Before Angular 20

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';

export const authGuard: CanActivateFn = (route, state) => {
  const routerService = inject(Router);

  const isLoggedIn = true;

  if (isLoggedIn) {
    return true;
  } else {
    routerService.navigate(['notFoundPage']);
    return false;
  }
};
```

Routing Guards

✓ Step 3: Add to Routes

ts

Copy Edit

```
// app.routes.ts
{
  path: 'dashboard',
  component: DashboardComponent,
  canActivate: [AuthGuard]
}
```